Object-Oriented Programming and Data Structures

COMP2012: Inheritance

Cecia Chan
Brian Mak
Pedro Sander

Department of Computer Science & Engineering
The Hong Kong University of Science and Technology
Hong Kong SAR, China

Example: University Admin Info

Let's implement a system for maintaining university administrative information.

- **Teacher** and **Student** are two completely separate classes.
- Their implementation uses separate code.
- However, some of their members and methods are implemented in the same way: name and department, and their handling member functions.
- Why would we implement the same function twice?
- That is not good re-use of software!

Example: U. Admin Info — Student Class

```
1  /* File: student1.h */
2  enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
3
4  class Student
5  {
6    private:
7      string name;
8      Department dept;
9      float GPA;
10     Course* enrolled;
11     int num_courses;
12
13   public:
14     Student(string n, Department d, float x) :
15         name(n), dept(d), GPA(x), enrolled(nullptr), num_courses(0) { }
16     string get_name() const;
17     Department get_department() const;
18     float get_GPA() const;
19     bool add_course(const Course& c);
20     bool drop_course(const Course& c);
21 };
```

Example: U. Admin Info — Teacher Class

```
/* File: teacher1.h */
enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
enum Rank { PROFESSOR, DEAN, PRESIDENT };

class Teacher
{
  private:
    string name;
    Department dept;
    Rank rank;
    string research_area;

  public:
    Teacher(string n, Department d, Rank r, string a) :
      name(n), dept(d), rank(r), research_area(a) { }
    string get_name() const;
    Department get_department() const;
    Rank get_rank() const;
    string get_research_area() const;
};
```

Things to Consider

- We want a way to say that **Student** and **Teacher** both have the same members: **name**, **dept**, but yet require them to keep a separate copy of these members.
- We want to share the code for **get_name** etc. between **Student** and **Teacher** as well.
- However, objects have states, and their consistency should be maintained when the objects' methods are called — so we cannot just write global functions to do it.

Solution#1: Re-use by Copying

Copy the code from one class to the other class, and change the class names.

- This is very error prone.
- It is also a maintenance nightmare.
  - What if we find a bug in the code in one class?
  - What if we want to improve the code? Perhaps we introduce a new member **address**.
- "Re-use by copying" is a bad idea!

Part I

What is Inheritance?

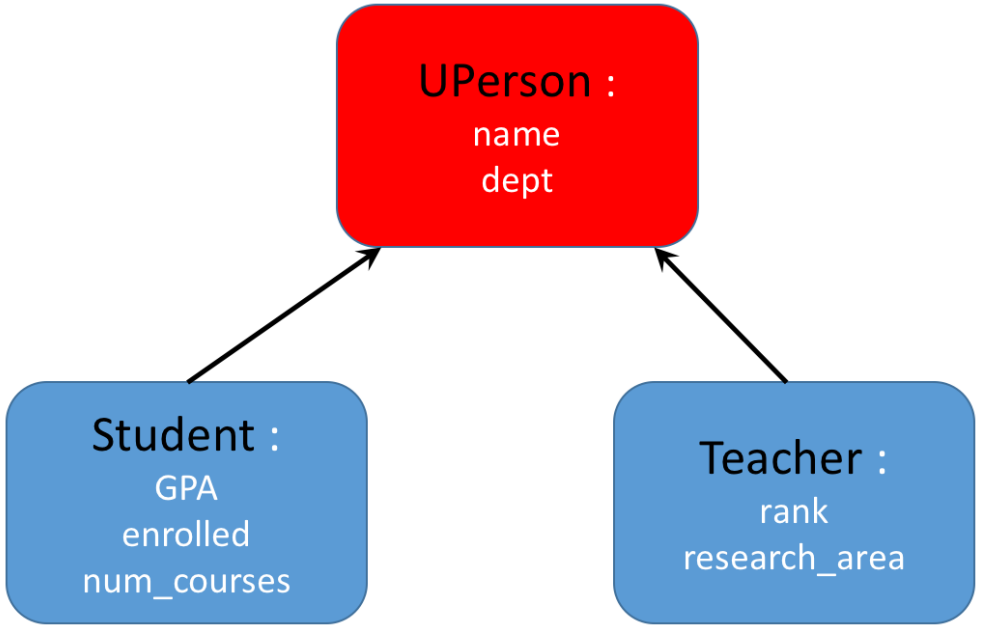
Solution#2: By Inheritance

Idea: Find out the common data members and member functions of **Student** and **Teacher** and put them into a parent class, called **UPerson** here, and apply the inheritance mechanism.

```
/* File: student1.h */
enum Department { CBME, CIVL, CSE, ECE, IELM,
MAE };

class Student
{
  private:
    string name;
    Department dept;
    float GPA;
    Course* enrolled;
    int num_courses;

  public:
    Student(string n, Department d, float x) :
        name(n), dept(d), GPA(x),
        enrolled(nullptr), num_courses(0) { }
    string get_name() const;
    Department get_department() const;
    float get_GPA() const;
    bool add_course(const Course& c);
    bool drop_course(const Course& c);
};
```

```
/* File: teacher1.h */
enum Department { CBME, CIVL, CSE, ECE, IELM,
MAE };
enum Rank { PROFESSOR, DEAN, PRESIDENT };

class Teacher
{
  private:
    string name;
    Department dept;
    Rank rank;
    string research_area;

  public:
    Teacher(string n, Department d, Rank r,
    string a) :
      name(n), dept(d), rank(r),
      research_area(a) { }
    string get_name() const;
    Department get_department() const;
    Rank get_rank() const;
    string get_research_area() const;
};
```

Solution#2: Inheritance — Base Class + Derived Classes



(Note: Only the data members are shown in each class.)

Solution#2: By Inheritance — UPerson Class

```
1  #ifndef UPERSON_H        /* File: uperson.h */
2  #define UPERSON_H
3
4  enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
5  class UPerson
6  {
7    private:
8      string name;
9      Department dept;
10
11   public:
12     UPerson(string n, Department d) : name(n), dept(d) { }
13     string get_name() const { return name; }
14     Department get_department() const { return dept; }
15 };
16
17 #endif
```

Solution#2: By Inheritance — Student Class

```
1  #ifndef STUDENT_H        /* File: student.h */
2  #define STUDENT_H
3
4  #include "uperson.h"     // Don't forget your parents!!
5  class Course { /* incomplete */ };
6
7  class Student : public UPerson  // Public inheritance
8  {
9    private:
10     float GPA;
11     Course* enrolled;
12     int num_courses;
13
14   public:
15     Student(string n, Department d, float x) :
16         UPerson(n, d), GPA(x), enrolled(nullptr), num_courses(0) { }
17     float get_GPA() const { return GPA; }
18     bool enroll_course(const string& c) { /* incomplete */ };
19     bool drop_course(const Course& c) { /* incomplete */ };
20 };
21 #endif
```

Solution#2: By Inheritance — Teacher Class

```
1  #ifndef TEACHER_H        /* File: teacher.h */
2  #define TEACHER_H
3
4  #include "uperson.h"     // Don't forget your parents!!
5  enum Rank { PROFESSOR, DEAN, PRESIDENT };
6
7  class Teacher : public UPerson  // Public inheritance
8  {
9    private:
10     Rank rank;
11     string research_area;
12
13   public:
14     Teacher(string n, Department d, Rank r, string a) :
15         UPerson(n, d), rank(r), research_area(a) { }
16     Rank get_rank() const { return rank; }
17     string get_research_area() const { return research_area; }
18 };
19
20 #endif
```

Inheritance

- Inheritance is the ability to define a new class based on an existing class with a hierarchy.
- The derived class inherits data members and member functions of the base class.
- New members and functions are added to the derived class.
- The new class only has to implement the behavior that is extra to the base class, and the code of the base class can be re-used in the derived class.
- In this example, **UPerson** is the base class, and **Student** and **Teacher** are the derived classes.
- **Student** and **Teacher** inherit all data members and functions from **UPerson**.
- E.g., data members of **Student** include the data members of **UPerson** {name, dept}, plus the extra data members declared in **Student**'s definition $\{\text{GPA}, \text{enrolled}, \text{num_courses}\}$.
- Inheritance enables code re-use.

Example: Inherited Members and Functions

```
1  #include <iostream>     /* File: inherited-fcn.cpp */
2  using namespace std;
3  #include "student.h"
4
5  void some_func(UPerson& uperson, Student& student) {
6      cout << uperson.get_name() << endl;
7      Department dept = uperson.get_department();
8      // Error! Base class object can't call derived class's function
9      uperson.enroll_course("COMP1001");
10
11     // Derived class object may call base class's member function
12     cout << student.get_name() << endl;
13     // Derived class object calls its own member functions
14     cout << student.get_GPA() << endl;
15     student.enroll_course("COMP2012");
16 }
17
18 int main() {
19     UPerson abby("Abby", CBME);
20     Student bob("Bob", CIVL, 3.0);
21     some_func(abby, bob);
22 }
```

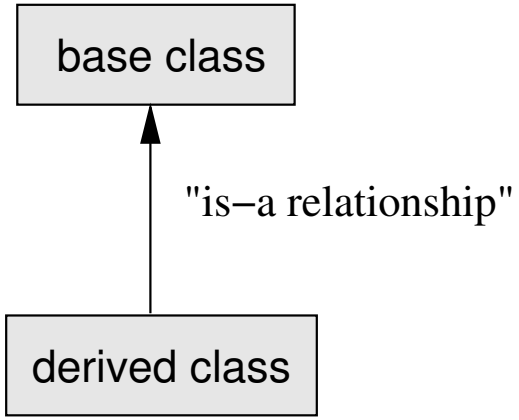
Polymorphic or Liskov Substitution Principle

Inheritance implements the is-a relationship.

- Since **Student** inherits from **UPerson**,
  - A **Student** object can be treated like a **UPerson** object.
  - All methods of **UPerson** can be called by a **Student** object.
- In other words, a **Student** object is a **UPerson** object.
- In general, an object of the derived class can be treated like an object of the base class under all circumstances.

If class **D** (a derived class) inherits from class **B** (the base class):

- Every **D** object is also a **B** object, but not vice-versa.
- **B** is a more general concept; **D** is a more specific concept.
- Wherever a **B** object is needed, a **D** object can be used instead.



Polymorphic or Liskov Substitution Principle ..

By deriving **Student** and **Teacher** classes are from **UPerson** class, we have:

- Since a **Student**/**Teacher** object is also a **UPerson** object, any functions defined on **UPerson** objects may also be called by any **Student** and **Teacher** objects.
- Obviously, functions defined on **UPerson** objects can only make use of **UPerson**'s data/functions; they can't use data/functions of **UPerson**'s derived classes which are not known yet at the time of their (**UPerson**) creation!

Function Expecting an Argument of Type	Will Also Accept
UPerson	**Student**
pointer to **UPerson**	pointer to **Student**
UPerson reference	**Student** reference

Example: Derived Objects Treated as Base Class Objects

```
1   #include <iostream>     /* File: print-label.cpp */
2   using namespace std;
3   #include "student.h"
4   #include "teacher.h"
5
6   void print_label(const UPerson& uperson)
7   {
8       cout << "Name: " << uperson.get_name() << endl;
9       cout << "Dept: " << uperson.get_department() << endl;
10  }
11
12  int main()
13  {
14      Student tom("Tom", CIVL, 3.9);
15      print_label(tom);   // Tom is also a UPerson
16
17      Teacher alan("Alan Turing", CSE, PROFESSOR, "AI");
18      print_label(alan);  // Alan is also a UPerson
19      return 0;
20  }
```

Example: Derived Objects Treated as Base Class Objects ..

```
#include <iostream>     /* File: print-label2.cpp */
using namespace std;
#include "student.h"

void print_label(const UPerson* uperson) {
    cout << "Name: " << uperson->get_name() << endl;
    cout << "Dept: " << uperson->get_department() << endl;
}
void print_label(const UPerson& uperson) {
    cout << "Name: " << uperson.get_name() << endl;
    cout << "Dept: " << uperson.get_department() << endl;
}
void print_label(const Student& student) {
    cout << "Name: " << student.get_name() << endl;
    cout << "Dept: " << student.get_department() << endl;
    cout << "GPA: " << student.get_GPA() << endl;
}
int main() {  // Which print_label()?
    Student tom("Tom", CIVL, 3.9); print_label(tom);
    UPerson& tom2 = tom; print_label(tom2);
    UPerson* p = &tom; print_label(p);
}
```

Quiz: Derived Objects Treated as Base Class Objects ..

```
#include <iostream>     /* File: substitute.cpp */
using namespace std;
#include "student.h"

int main() {
    void dance(const UPerson& p); // Anyone can dance
    void dance(const UPerson* p); // Anyone can dance
    void study(const Student& s); // Only students study
    void study(const Student* s); // Only students study
    UPerson p("P", IELM); Student s("S", MAE, 3.3);

    // Which of the following statements can compile?
    dance(p);
    dance(s);
    dance(&p);
    dance(&s);
    study(s);
    study(p);
    study(&s);
    study(&p);
}
```

Extending Class Hierarchy

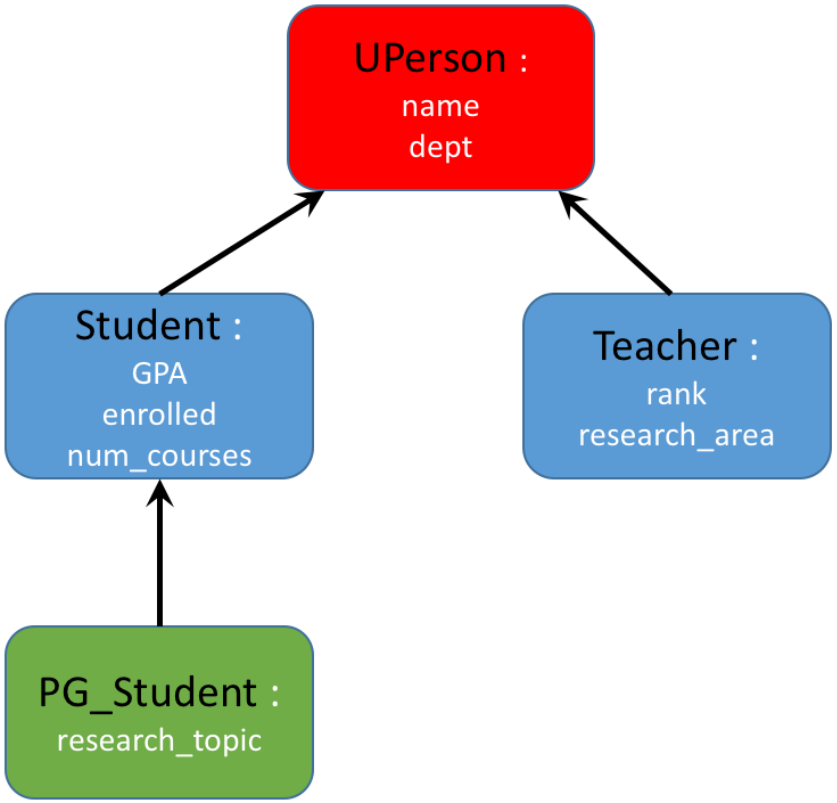
We can easily add classes to our existing class hierarchy of **UPerson**, **Student**, and **Teacher**.

- New classes can immediately benefit from all functions that are available to their base classes.
- e.g., void print_label(const UPerson& person) will work immediately for a new class called **PG_Student**, even though this type of objects was unknown when **print_label( )** was designed and written.
- In fact, it is not even necessary to recompile the existing code: It is enough to link the new class with the object codes of **UPerson** and **print_label( )**.
- Advanced use: Link in new objects while the code is running!

Direct and Indirect Inheritance

Let's add a new class **PG_Student** to the hierarchy.

- **PG_Student** is directly derived from **Student**.
- It is indirectly derived from **UPerson**.
- So a **PG_Student** object is also a **UPerson** object.
- **UPerson** is called an indirect base class of **PG_Student**.



Direct and Indirect Inheritance — PG_Student Class

```
1  #ifndef PG_STUDENT_H    /* File: pg-student.h */
2  #define PG_STUDENT_H
3
4  #include "student.h"
5
6  class PG_Student : public Student
7  {
8    private:
9      string research_topic;
10
11   public:
12     PG_Student(string n, Department d, float x) :
13         Student(n, d, x), research_topic("") { }
14
15     string get_topic() const { return research_topic; }
16     void set_topic(const string& x) { research_topic = x; }
17 };
18
19 #endif
```

Example: Indirect Inheritance

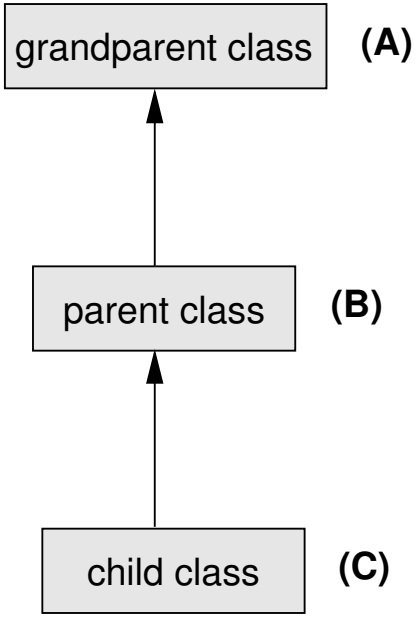
- Let's promote Tom to **PG_Student**.
- Can Tom still use the **print_label( )** function?

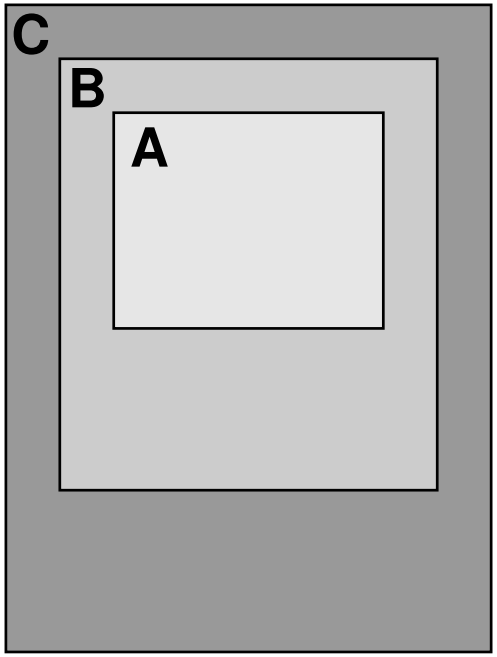
```
1  #include <iostream>     /* File: pg-print-label.cpp */
2  using namespace std;
3  #include "pg-student.h" // Change student.h to pg-student.h
4
5  void print_label(const UPerson& uperson)
6  {
7      cout << "Name: " << uperson.get_name() << endl;
8      cout << "Dept: " << uperson.get_department() << endl;
9  }
10
11 int main()
12 {
13     PG_Student tom("Tom", CIVL, 3.9); // Tom is now a PG Student
14     print_label(tom);                 // Tom is also a UPerson
15     return 0;
16 }
```

Part II

Initialization of Classes in an Inheritance Hierarchy

Initialization of Base Class Objects





- If class C is derived from class B which is in turn derived from class A, then C will contain data members of both B and A.
- Class C's constructor can only call class B's constructor, and class B's constructor can only call class A's constructor.
- It is the responsibility of each derived class to initialize its direct base class correctly.

Initialization of Base Class Objects by Initializers

- Before a **Student** object can come into existence, we have to create its **UPerson** parent first.
- **Student**'s constructors have to call a **UPerson**'s constructor through the member initializer list.

```
Student::Student(string n, Department d, float x) :
  UPerson(n,d), GPA(x), enrolled(nullptr), num_courses(0) { }
```

- Similarly, **PG_Student** has to create its **Student** part before it can be created.
- But, it does not need to create its **UPerson** part directly by calling **UPerson**'s constructor.
- In fact, its **UPerson** part should have been created by **Student**.

```
PG_Student::PG_Student(string n, Department d, float x) :
    Student(n, d, x), research_topic("") { }
```

Order of Cons/Destruction: Student w/ an Address

```
1   #include <iostream>     /* File: init-order.cpp */
2   using namespace std;
3
4   class Address {
5     public:
6       Address() { cout << "Address's constructor" << endl; }
7       ~Address() { cout << "Address's destructor" << endl; }
8   };
9
10  class UPerson {
11    public:
12      UPerson() { cout << "UPerson's constructor" << endl; }
13      ~UPerson() { cout << "UPerson's destructor" << endl; }
14  };
15
16  class Student : public UPerson {
17    public:
18      Student() { cout << "Student's constructor" << endl; }
19      ~Student() { cout << "Student's destructor" << endl; }
20
21    private: Address address;
22  };
23
24  int main() { Student x; return 0; }
```

Order of Cons/Destruction: Student w/ an Address ..

```
UPerson's constructor
Address's constructor
Student's constructor
Student's destructor
Address's destructor
UPerson's destructor
```

That is, the order is construction of a class object

1. its parent
2. its data members
   (in the order of their appearance in the class definition)
3. itself

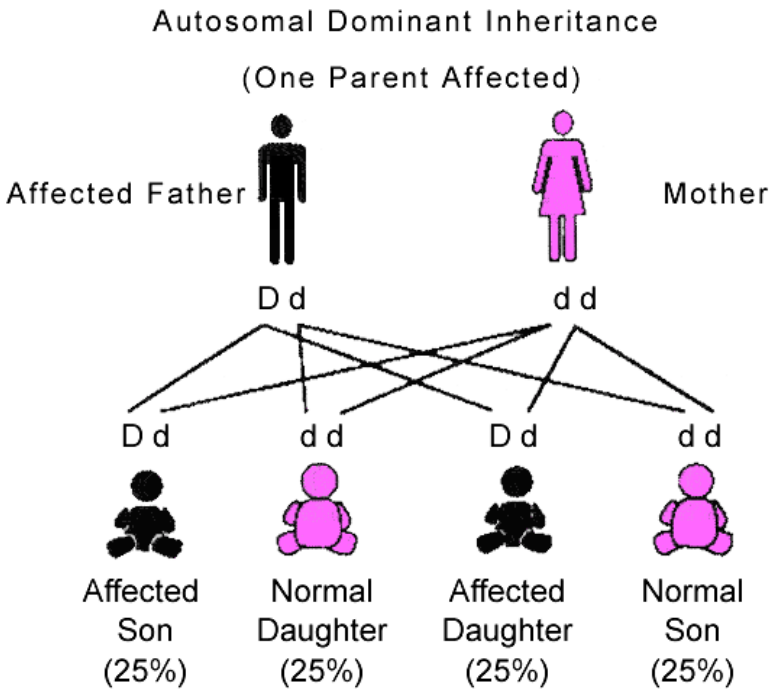
Order of Cons/Destruction: Move Address to UPerson

```
1   #include <iostream>     /* File: init-order2.cpp */
2   using namespace std;
3
4   class Address {
5     public:
6       Address() { cout << "Address's constructor" << endl; }
7       ~Address() { cout << "Address's destructor" << endl; }
8   };
9
10  class UPerson {
11    public:
12      UPerson() { cout << "UPerson's constructor" << endl; }
13      ~UPerson() { cout << "UPerson's destructor" << endl; }
14    private: Address address;
15  };
16
17  class Student : public UPerson {
18    public:
19      Student() { cout << "Student's constructor" << endl; }
20      ~Student() { cout << "Student's destructor" << endl; }
21  };
22
23  int main() { Student x; return 0; }
```

Question: What is the output now?

Part III

Some Problems of Inheritance



Problem #1: Slicing

- An assignment from a derived class object to a base class object results in "slicing".
- This is rarely desirable.
- Once slicing has happened, there is no trace of the fact that we started with a derived class.

```
1  #include <iostream>     /* File: slice.cpp */
2  #include <string>
3  using namespace std;
4  #include "../basics/uperson.h"
5  #include "../basics/student.h"
6
7  int main()
8  {
9      Student student("Snoopy", CSE, 3.5);
10     UPerson* pp = &student;
11     UPerson* pp2 = new Student("Mickey", ECE, 3.4);
12
13     UPerson uperson("Unknown", CIVL);
14     uperson = student; // What does "uperson" have?
15     delete static_cast<Student*>(pp2);
16     return 0;
17 }
```

Problem #2: Name Conflicts

```
/* File: name-conflict.h */
void print(int x, int y) { cout << x << " , " << y << endl; }

class B
{
  private:
    int x, y;
  public:
    B(int p = 1, int q = 2) : x(p), y(q)
        { cout << "Base class constructor: "; print(x, y); }
    void f() const { cout << "Base class: "; print(x, y); }
};

class D : public B
{
  private:
    float x, y;
  public:
    D() : x(10.2), y(20.6) { cout << "Derived class constructor\n"; }
    void f() const { cout << "Derived class: "; print(x, y);B::f(); }
}; // D::f() hides away B::f(). This case is NOT function overloading
```

Problem #2: Name Conflicts ..

```
1  #include <iostream>     /* File: name-conflict.cpp */
2  using namespace std;
3  #include "name-conflict.h"
4
5  void smart(const B* p) { cout << "Inside smart(): "; p->f(); }
6
7  int main()
8  {
9      B base(5, 6); cout << endl;
10     D derive; cout << endl;
11
12     B* bp = &base; bp->f(); cout << endl;
13     D* dp = &derive; dp->f(); cout << endl;
14
15     bp = &derive; bp->f(); cout << endl;
16
17     cout << "Call smart(bp): "; smart(bp);
18     cout << "Call smart(dp): "; smart(dp);
19     return 0;
20 }
```

Problem #2: Name Conflicts Output

```
Base class constructor: 5 , 6

Base class constructor: 1 , 2
Derived class constructor

Base class: 5 , 6

Derived class: 10 , 20
Base class: 1 , 2

Base class: 1 , 2

Call smart(bp): Inside smart(): Base class: 1 , 2
Call smart(dp): Inside smart(): Base class: 1 , 2
```

- `D::f()` hides away `B::f()`.
- This is **not** overloading because the 2 functions are in different class scopes, `D::` and `B::`, respectively.

Summary

- Behavior and structure of the base class is inherited by the derived class.
- However, constructors and destructor are an exception. They are never inherited.
- There is a kind of contract between a base class and a derived class:
  - The base class provides functionality and structure (methods and data members).
  - The derived class guarantees that the base class is initialized in a consistent state by calling an appropriate constructor.
- A base class is constructed before the derived class.
- A base class is destructed after the derived class.

Part IV

Access Control: public, protected, private

Example: Add **print( )** to UPerson/Student Class

```
/* File: print1.cpp */

class UPerson { public: void print() const; ...  };

class Student: public UPerson { public: void print() const; ...  };

void UPerson::print() const
{
    cout << "--- UPerson details ---" << endl;
    cout << "Name: " << name << endl << "\nDept: " << dept << endl;
}

void Student::print() const
{
    cout << "--- Student details ---" << endl
         << "Name: " << name << endl
         << "\nDept: " << dept << endl << "Enrolled in:" << endl;
    for (int i = 0; i < num_courses; i++)
        enrolled[i].print(); // Assume a print function in Course
}
```

Example: Student::print( ) Doesn't Compile!

- The implementation of **Student::print( )** given before doesn't work. It will raise an error during compilation:

  Student::print(): **name** and **dept** are declared private.

- **name** is a private data member of the base class **UPerson**.
- Public inheritance does not change the access control of the data members of the base class.
- Private members are still only available to base class' own member functions (methods), and not to any other classes including derived classes (except friends) or global functions.

One Solution: Protected Data Members

```
class UPerson              /* File: protected-uperson.h */
{
  protected:
    string name;
    Department dept;

  public:
    UPerson(string n, Department d) : name(n), dept(d) { }
    void print() const;
    ...
};
```

- By making **name** and **dept** protected, they are accessible to methods in the base class as well as methods in the derived classes.
- They should not be public though!
  (Principle of information hiding.)

Member Access Control: public, protected, private

There are 3 levels of member (data or methods) access control:

1. public: accessible to
   - member functions of the class (from class developer)
   - any member functions of other classes (application programmers)
   - any global functions (application programmers)

2. protected: accessible to
   - member functions and friends of the class
   - member functions and friends of its derived classes (subclasses)

   ⇒ class developer restricts what subclasses may directly use

3. private: accessible only to
   - member functions and friends of the class

   ⇒ class developer enforces information hiding

Without inheritance, private and protected control are the same.

protected vs. private

So why not always use protected instead of private?

- Because protected means that we have less data encapsulation: Remember that all derived classes can access protected data members of the base class.
- Assume that later you decided to change the implementation of the base class having the protected data members.
- For example, we might want to represent dept of **UPerson** by a new class called **class Department** instead of **enum Department**. If the **dept** data member is private, we can easily make this change. The update on the **UPerson** class documentation is small.
- However, if it is protected, we have to go through not only the **UPerson** class, but also all its derived classes and change them. We also need to update the documentation of many classes.

protected vs. private ..

- In general, it is preferable to have private members instead of protected members.
- Use protected only where it is really necessary. private is the only category ensuring full data encapsulation.
- This is particularly true for data members, but it is less harmful to have protected member functions. Why?
- When a class has protected members, it is a hint that it expects others to derive sub-classes from it.

In our example, there is no reason at all to make **name**, and **dept** protected, as we can access the name and address through appropriate public member functions.

Write Student::print( ), Teacher::print( ) with UPerson's Public Member Functions Only

```
void Student::print() const /* correct-student-print.cpp */
{
    cout << "--- Student details ---" << endl
         << "Name: " << get_name() << endl  // Use UPerson's public fcn
         << "Dept: " << get_dept() << endl  // Use UPerson's public fcn
         << "Enrolled in:" << endl;

    for (int i = 0; i < num_courses; i++)
        enrolled[i].print();                // Use Course's public fcn
}
```

```
void Teacher::print() const /* correct-teacher-print.cpp */
{
    cout << "--- Teacher details ---" << endl
         << "Name: " << get_name() << endl  // Use UPerson's public fcn
         << "Dept: " << get_dept() << endl  // Use UPerson's public fcn
         << "Rank: " << get_rank() << endl; // Use its own fcn
}
```

Write Student::print( ), Teacher::print( ) with UPerson's Public Member Functions Only ..

Let's use the new **print( )** functions now.

```
/* File: print-example.cpp (incomplete) */
UPerson newton("Isaac Newton", MAE);
Teacher turing("Alan Turing", CSE, DEAN);
Student edison("Thomas Edison", ECE, 2.5);
edison.enroll_course("COMP2012");

newton.print();
turing.print();
edison.print();
```

Write Student::print( ), Teacher::print( ) with UPerson's Public Member Functions Only — Expected Output

```
--- UPerson details ---
Name: Isaac Newton
Dept: 5

--- Teacher details ---
Name: Alan Turing
Dept: 2
Rank: 1

--- Student details ---
Name: Thomas Edison
Dept: 3
Enrolled in:
COMP2012
```

Part V

Polymorphism: Dynamic Binding & Virtual Function

Global **print( )** for UPerson and its Derived Objects

```
1  #include <iostream>     /* File: print-label.cpp */
2  using namespace std;
3  #include "student.h"
4  #include "teacher.h"
5
6  void print_label_v(UPerson uperson) { uperson.print(); }
7  void print_label_r(const UPerson& uperson) { uperson.print(); }
8  void print_label_p(const UPerson* uperson) { uperson->print(); }
9
10 int main() {
11     UPerson uperson("Charlie Brown", CBME);
12     Student student("Edison", ECE, 3.5);
13     Teacher teacher("Alan Turing", CSE, PROFESSOR, "CS Theory");
14     student.add_course("COMP2012"); student.add_course("MATH1003");
15
16     cout << "\n##### PASS BY VALUE  #####\n";
17     print_label_v(uperson); print_label_v(student); print_label_v(teacher);
18
19     cout << "\n##### PASS BY REFERENCE  #####\n";
20     print_label_r(uperson); print_label_r(student); print_label_r(teacher);
21
22     cout << "\n##### PASS BY POINTER  #####\n";
23     print_label_p(&uperson); print_label_p(&student); print_label_p(&teacher);
24 }
```

Are These Outputs What You Want?

```
##### PASS BY VALUE  #####
--- UPerson Details ---
Name: Charlie Brown
Dept: 0
--- UPerson Details ---
Name: Edison
Dept: 3
--- UPerson Details ---
Name: Alan Turing
Dept: 2
```

```
##### PASS BY POINTER  #####
--- UPerson Details ---
Name: Charlie Brown
Dept: 0
--- UPerson Details ---
Name: Edison
Dept: 3
--- UPerson Details ---
Name: Alan Turing
Dept: 2
```

```
##### PASS BY REFERENCE  #####
--- UPerson Details ---
Name: Charlie Brown
Dept: 0
--- UPerson Details ---
Name: Edison
Dept: 3
--- UPerson Details ---
Name: Alan Turing
Dept: 2
```

You Probably Want This

```
#####  PASS BY VALUE  #####
--- UPerson Details ---
Name: Charlie Brown
Dept: 0
--- UPerson Details ---
Name: Edison
Dept: 3
--- UPerson Details ---
Name: Alan Turing
Dept: 2

#####  PASS BY REFERENCE  #####
--- UPerson Details ---
Name: Charlie Brown
Dept: 0
--- Student Details ---
Name: Edison
Dept: 3
2 Enrolled courses: COMP2012 MATH1003
--- Teacher Details ---
Name: Alan Turing
Dept: 2
Rank: 0
Research area: CS Theory
```

```
#####  PASS BY POINTER  #####
--- UPerson Details ---
Name: Charlie Brown
Dept: 0
--- Student Details ---
Name: Edison
Dept: 3
2 Enrolled courses: COMP2012 MATH1003
--- Teacher Details ---
Name: Alan Turing
Dept: 2
Rank: 0
Research area: CS Theory
```

Static (or Early) Binding

- Because of the polymorphic substitution principle, a function accepting a base class object also accepts its derived objects.
- In our current case, the following 3 global print functions:

```
void print_label_v(UPerson uperson) { uperson.print(); }
void print_label_r(const UPerson& uperson) { uperson.print(); }
void print_label_p(const UPerson* uperson) { uperson->print(); }
```

  will accept objects of **UPerson**/**Student**/**Teacher** classes, and objects derived from them directly or indirectly.
- However, when these function codes are compiled, the compiler only looks at the static type of **uperson** which is **UPerson**, **const UPerson&**, or **const UPerson***, and the method **UPerson::print( )** is called.
- Static binding: the binding (association) of a function name (here **print( )**) to the appropriate method is done by a static analysis of the code at compile time based on the static (or declared) type of the object (here, **UPerson**) making the call.

Static Binding: Who May Call Whose **print( )**?

```
1   #include <iostream>     /* File: static-example.cpp */
2   using namespace std;
3   #include "teacher.h"
4
5   int main()
6   {
7       UPerson uperson("Charlie Brown", CBME);
8       Teacher teacher("Alan Turing", CSE, PROFESSOR, "CS Theory");
9       UPerson* u; Teacher* t;
10
11      cout << "\nUPerson object pointed by UPerson pointer:\n";
12      u = &uperson; u->print();
13
14      cout << "\nTeacher object pointed by Teacher pointer:\n";
15      t = &teacher; t->print();
16
17      cout << "\nTeacher object pointed by UPerson pointer:\n";
18      u = &teacher; u->print();
19
20      cout << "\nUPerson object pointed by Teacher pointer:\n";
21      t = &uperson; t->print(); // Error: convert base-class ptr
22                                //        to derived-class ptr
23      t = static_cast<Teacher*>(&uperson); t->print(); // Ok, but ...
24  }
```

Dynamic (or Late) Binding

- By default, C++ uses static binding. (Same as C, Pascal, and FORTRAN.)
- In static binding, what a pointer really points to, or what a reference actually refers to is not considered; only the pointer/reference type is.
- But C++ also allows dynamic binding which is supported through virtual functions.
- When dynamic binding is used, the actual method to be called is selected using the actual type of the object in the call, but only if the object is passed by reference or pointer. i.e.,

  print_label_r(a **UPerson** object**)** calls **UPerson::print( )**;
  print_label_r(a **Teacher** object**)** calls **Teacher::print( )**;
  print_label_r(a **Student** object**)** calls **Student::print( )**.
- Magic: the possible object types don't need to be known at the time when the function definition is being compiled!!!

Virtual Functions

- A virtual function is declared using the keyword virtual in the class definition, and not in the method implementation, if it is defined outside the class.
- Once a method is declared virtual in the base class, it is automatically virtual in all directly or indirectly derived classes.
- Even though it is not necessary to use the virtual keyword in the derived classes, it is a good style to do so because it improves the readability of header files.
- Calls to virtual functions are a little bit slower than normal function calls. The difference is extremely small and it is not worth worrying about, unless you write very speed-critical code.

Virtual Function: UPerson Class

```
1  #ifndef V_UPERSON_H     /* File: v-uperson.h */
2  #define V_UPERSON_H
3
4  enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
5  class UPerson
6  {
7    private:
8      string name;
9      Department dept;
10
11   public:
12     UPerson(string n, Department d) : name(n), dept(d) { }
13     string get_name() const { return name; }
14     Department get_department() const { return dept; }
15
16     virtual void print() const
17     {
18         cout << "--- UPerson Details --- \n"
19              << "Name: " << name << "\nDept: " << dept << "\n";
20     }
21 };
22 #endif
```

Virtual Function: Course Class

```
1   #ifndef COURSE_H        /* File: course.h */
2   #define COURSE_H
3
4   class Course
5   {
6     private:
7       string code;
8
9     public:
10      Course(const string& s) : code(s) { }
11      ~Course() { cout << "destruct course: " << code << endl; }
12      void print() const { cout << code; }
13  };
14
15  #endif
```

Virtual Function: Student Class

```
1  #ifndef V_STUDENT_H     /* File: v-student.h */
2  #define V_STUDENT_H
3  #include "course.h"
4  #include "v-uperson.h"
5
6  class Student : public UPerson {  // Public inheritance
7    private:
8      float GPA; Course* enrolled[50]; int num_courses;
9
10   public:
11     Student(string n, Department d, float x) :
12         UPerson(n, d), GPA(x), num_courses(0) { }
13     ~Student() { for (int j = 0; j < num_courses; ++j) delete enrolled[j]; }
14     float get_GPA() const { return GPA; }
15     bool add_course(const string& s)
16         { enrolled[num_courses++] = new Course(s); return true; }
17     virtual void print() const {
18         cout << "--- Student Details --- \n"
19              << "Name: " << get_name() << "\nDept: " << get_department()
20              << "\n" << num_courses << " Enrolled courses: ";
21         for (int j = 0; j < num_courses; ++j)
22            { enrolled[j]->print(); cout << ' '; } cout << endl;
23     }
24 };
25 #endif
```

Virtual Function: Teacher Class

```
1  #ifndef V_TEACHER_H     /* File: v-teacher.h */
2  #define V_TEACHER_H
3  #include "v-uperson.h"
4
5  enum Rank { PROFESSOR, DEAN, PRESIDENT };
6  class Teacher : public UPerson  // Public inheritance
7  {
8    private:
9      Rank rank;
10     string research_area;
11
12   public:
13     Teacher(string n, Department d, Rank r, string a) :
14         UPerson(n, d), rank(r), research_area(a) { }
15     Rank get_rank() const { return rank; }
16     string get_research_area() const { return research_area; }
17     virtual void print() const {
18         cout << "--- Teacher Details --- \n"
19              << "Name: " << get_name()
20              << "\nDept: " << get_department()
21              << "\nRank: " << rank
22              << "\nResearch area: " << research_area << endl;
23     }
24 };
25 #endif
```

Polymorphism

Polymorphism

poly = multiple *morphos* = shape

- Polymorphism in C++ means that we can work with objects without knowing their precise type at compile time.
- `void print_label_p(const UPerson* uperson) { uperson->print(); }`
  The type of the object pointed to by **uperson** is not known to the programmer writing this code, nor to the compiler.
- We say that **uperson** exhibits polymorphism, because the object can take on multiple "shapes" (Student, Teacher, PG_Student, etc.).
- Polymorphism allows us to write programs that behave correctly even when used with objects of derived classes.
- Again a pointer or reference must be used to take advantage of polymorphism.

Question: Why won't polymorphism work if pass-by-value is used?

Example: Polymorphism using Virtual Function

```
1   #include <iostream>     /* File: v-example.cpp */
2   using namespace std;
3   #include "v-student.h"
4   #include "v-teacher.h"
5
6   int main()
7   {
8       char person_type; string name; UPerson* uperson[3];
9
10      for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
11      {
12          cout << "Input the uperson type (u/s/t) and his name : ";
13          cin >> person_type >> name;
14          switch (person_type)
15          {
16              case 'u': uperson[j] = new UPerson(name, MAE); break;
17              case 's': uperson[j] = new Student(name, CIVL, 4.0); break;
18              case 't': uperson[j] = new Teacher(name, CSE, DEAN, "AI"); break;
19          }
20      }
21
22      for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
23          uperson[j]->print();
24      return 0;
25  } // The example does't destruct the dynamically allocated objects
```

Run-Time Type Information (RTTI)

- RTTI is a run-time facility that keeps track of dynamic types ⇒ program can determine an object's type at run-time.
- **typeid(<expression>)** returns an object of the type **type_info**. It has the member function

  ```
  const char* name() const
  ```

  that returns the type name of the expression.
- `typeid()` can be used to find out the type name of any expression (which can be an object, pointer, reference, etc.)
- **static_cast( )** may be used to perform type conversions,
  - including conversions between pointers to classes in an inheritance hierarchy;
  - it doesn't consult RTTI to ensure the conversion is safe;
  - thus, it runs faster.

RTTI ..

- **dynamic_cast( )**, on the other hand,
  - only works on pointers and references of polymorphic class (with virtual functions) types;
  - consults RTTI to make sure the conversion result refers to a valid complete object of the target type.
  - If the input is a pointer: it returns a pointer to a valid complete object of the target type, otherwise, a nullptr.
  - If the input is a reference: it returns a reference to a valid complete object of the target type, otherwise, it is a runtime error!

Example: RTTI typeid(<expression>).name()

```
1   #include <iostream>     /* File: rtti.cpp */
2   #include <typeinfo>     // In order to use typeid
3   using namespace std;
4   #include "v-student.h"
5   #include "v-teacher.h"
6   int main()
7   {
8       char person_type; string name; UPerson* uperson[3];
9
10      for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
11      {
12          cout << "Input the uperson type (u/s/t) and his name : ";
13          cin >> person_type >> name;
14          switch (person_type)
15          {
16              case 'u': uperson[j] = new UPerson(name, MAE); break;
17              case 's': uperson[j] = new Student(name, CIVL, 4.0); break;
18              case 't': uperson[j] = new Teacher(name, CSE, DEAN, "AI"); break;
19          }
20      }
21
22      for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
23          cout << "The uperson #" << j << " is a "
24               << typeid(*uperson[j]).name() << endl; // RTTI
25  } // The example does't destruct the dynamically allocated objects
```

Example: RTTI **typeid(<expression>).name()** Output

```
Input the uperson type (s/t) and his name : s Abby
Input the uperson type (s/t) and his name : t Brian
Input the uperson type (s/t) and his name : s Chris
The uperson #0 is a 7Student
The uperson #1 is a 7Teacher
The uperson #2 is a 7Student
```

- The returned type name is implementation dependent.
- i.e., different compilers may give different printout.
- In this course, we assume the above printout from the g++ compiler.

Example: Compare **typeid(<expression>)**

```
1   #include <iostream>     /* File: rtti2.cpp */
2   #include <typeinfo>     // In order to use typeid
3   using namespace std;
4   #include "v-student.h"
5   #include "v-teacher.h"
6   int main()
7   {
8       char person_type; string name; UPerson* uperson[3];
9
10      for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j) {
11          cout << "Input the uperson type (u/s/t) and his name : ";
12          cin >> person_type >> name;
13          switch (person_type) {
14              case 'u': uperson[j] = new UPerson(name, MAE); break;
15              case 's': uperson[j] = new Student(name, CIVL, 4.0); break;
16              case 't': uperson[j] = new Teacher(name, CSE, DEAN, "AI"); break;
17          }
18      }
19      cout << boolalpha;
20      for (int i = 0; i < sizeof(uperson)/sizeof(UPerson*); ++i) {
21          for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
22              cout << (typeid(*uperson[i]) == typeid(*uperson[j])) << "\t";
23          cout << endl;
24      }
25  } // The example does't destruct the dynamically allocated objects
```

Different Ways to Check Real Identity of Objects

1. Compare directly the returned `typeid()` objects.

   ```
   typeid(*uperson[i]) == typeid(*uperson[j])
   ```

   `typeid()` can be used to find the real identity from a base pointer only if,

   - the base class is polymorphic (having virtual function(s))
   - the expression represents an object (not pointer)

2. Compare the names of the returned `typeid()` objects.

   ```
   strcmp(typeid(*uperson[i]).name(),
          typeid(*uperson[j]).name()) == 0
   ```

3. Try dynamic-casting the object pointer to the desired type and check if it succeeds, though it also succeeds when the cast type is a pointer to an ancestor of the actual type.

   ```
   dynamic_cast<Student*>(uperson[i]) != nullptr
   ```

Overriding and Virtual Functions

- When a derived class defines a method with the same name as a base class method, it overrides the base class method. e.g.

  Student::print( ) overrides **UPerson::print( )**

- This is necessary if the behaviour of the base class method is not good enough for derived classes.
- All derived classes should respond to the same request (**print**!), but their response varies depending on the object.
- The designer of a base class (**UPerson**) must realize that this is necessary, and declare its **print( )** a virtual function.
- Overriding is not possible if the method is not virtual.
- For overriding to work, the prototype of the virtual function in the derived class must be identical to that of the base class. To safeguard this, C++11 recommends a new keyword override in the function declaration in the derived classes.

```
/* in derived classes: Student or Teacher, etc. */
virtual void print() const override;
```

C++11 Keyword: override

```
1   #include <iostream>     /* File: override.cpp */
2   using namespace std;
3
4   class Base
5   {
6     public:
7       virtual void f(int a) const { cout << a << endl; }
8   };
9
10  class Derived: public Base
11  {
12      int x {25};
13    public:
14      void f(int) const override;
15  };
16
17  // Don't repeat the keyword override here
18  void Derived::f(int b) const { cout << x+b << endl; }
19
20  int main() { Derived d; Base& b = d; b.f(5); return 0; }
```

Virtual Functions vs. Non-Virtual Functions

- The designer of the base class must distinguish carefully between two kinds of methods:
  - If the method works exactly the same for all derived classes, it should not be a virtual function.
  - If the precise behaviour of the method depends on the object, it should be a virtual function.
- However, derived classes have to be careful in implementing such methods because of the substitution principle. The "effect" (meaning) of calling the derived class method must be the "same" as that for the base class method.
- Overriding is for specializing a behaviour, not changing the semantics. E.g., **print( )** should not be a method that does something completely different.
- The compiler can only check that overriding is done syntactically correctly, not whether the semantics of the method is preserved.

Overloading vs. Overriding

Overloading

Allows programmers to use functions with the same name, but different arguments for similar purposes.

- The decision on which function to use — overload resolution — is done by the compiler when the program is compiled.
- There is no dynamic binding.

Overloading vs. Overriding ..

Overriding

Allows a derived class to provide a different implementation for a function declared in the base class.

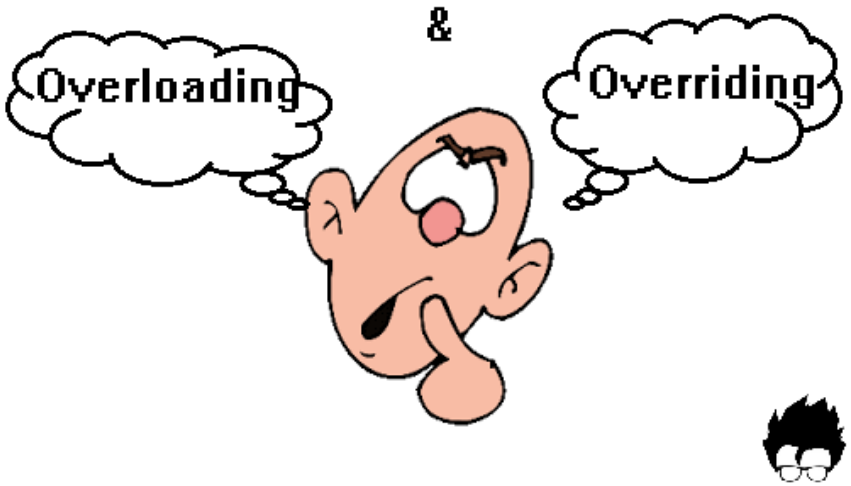
- Overriding is only possible with inheritance and dynamic binding — without inheritance there is no overriding.
- The decision of which method to use is done at the moment that the method is called.
- It only applies to member methods, not global functions.

Question: Can a virtual function declared as protected or private in the derived classes (while it is declared as public in the base class)?

E.g., for the **UPerson** example, try to declare the print function as protected or private in **Student** or **Teacher**.

Part VI

Virtual Functions and Destructors & Constructors



Example: Destruction with No Substitution

```
1   #include <iostream>     /* File: concrete-destructors.cpp */
2   using namespace std;
3   #include "v-student.h"
4
5   int main()
6   {
7       UPerson* p = new UPerson("Adam", ECE);
8       delete p;
9
10      Student* s = new Student("Simpson", CSE, 3.8);
11      s->add_course("COMP1021");
12      s->add_course("COMP2012");
13      delete s;
14  }
```

- **delete p** will call **UPerson**'s destructor, and **delete s** will call **Student**'s destructor respectively. Everything works fine.

Example: Destruction with Substitution

```
1  #include <iostream>     /* File: require-v-destructors.cpp */
2  using namespace std;
3  #include "v-student.h"
4
5  int main()
6  {
7      Student* s = new Student("Simpson", CSE, 3.8);
8      s->add_course("COMP1021");
9      s->add_course("COMP2012");
10
11     UPerson* p = s;
12     delete p; // Can we call UPerson's destructor on a Student?
13 }
```

- Here **p** actually points to a **Student** object.
- **delete p** calls the **UPerson**'s destructor, and not **Student**'s destructor.
- The behavior of destructing a derived class object by its base class destructor is undefined!

Virtual Destructor

- The solution is again using dynamic binding, and making the destructors virtual.

```
class UPerson                          /* File: v-uperson2.h */
{
  public: virtual ~UPerson() = default;
  ...
};

class Student : public UPerson      /* File: v-student2.h */
{
  public:
    virtual ~Student()
    {
        for (int j = 0; j < num_courses; ++j)
            delete enrolled[j];
    }
  ...
};
```

Virtual Destructor ..

```
1  #include <iostream>     /* File: v-destructors.cpp */
2  using namespace std;
3  #include "v-student2.h" // With virtual destructor
4
5  int main()
6  {
7      Student* s = new Student("Simpson", CSE, 3.8);
8      s->add_course("COMP1021");
9      s->add_course("COMP2012");
10
11     UPerson* p = s;
12     delete p;           // Actually call Student's destructor
13 }
```

- Now, **delete p** correctly calls the **Student**'s destructor if **p** points to a **Student** object.
- When a class does not have a virtual destructor, it is a strong hint that the class is not designed to be used as a base class.

Example: Order of Constructions and Destruction

```
1  #include <iostream>     /* File: construction-destruction-order.cpp */
2  using namespace std;
3
4  class Base
5  {
6    public:
7      Base() { cout << "Base's constructor\n"; }
8      ~Base() { cout << "Base's destructor\n"; }
9  };
10
11 class Derived : public Base
12 {
13   public:
14     Derived() { cout << "Derived's constructor\n"; }
15     ~Derived() { cout << "Derived's destructor\n"; }
16 };
17
18 int main()
19 {
20     Base* p = new Derived;
21     delete p;
22 }
```

Question: What is the output?

Example: Order of Constructions and Destruction ..

Question: What is the output when virtual destructors are used?

```
1  #include <iostream>     /* File: construction-v-destruction-order.cpp */
2  using namespace std;
3
4  class Base
5  {
6    public:
7      Base() { cout << "Base's constructor\n"; }
8      virtual ~Base() { cout << "Base's destructor\n"; }
9  };
10
11 class Derived : public Base
12 {
13   public:
14     Derived() { cout << "Derived's constructor\n"; }
15     virtual ~Derived() { cout << "Derived's destructor\n"; }
16 };
17
18 int main()
19 {
20     Base* p = new Derived;
21     delete p;
22 }
```

Example: Calling Virtual Functions in Constructors

```
1   #include <iostream>     /* File: construct-vf.cpp */
2   using namespace std;
3
4   class Base {
5     public:
6       Base() { cout << "Base's constructor\n"; f(); }
7       virtual void f() { cout << "Base::f()" << endl; }
8   };
9
10  class Derived : public Base {
11    public:
12      Derived() { cout << "Derived's constructor\n"; }
13      virtual void f() override { cout << "Derived::f()" << endl; }
14  };
15
16  int main() {
17      Base* p = new Derived;
18      cout << "Derived-class object created" << endl;
19      p->f();
20  }
```

Example: Calling Virtual Functions in Constructors ..

The output is:

```
Base's constructor
Base::f( )
Derived's constructor
Derived-class object created
Derived::f( )
```

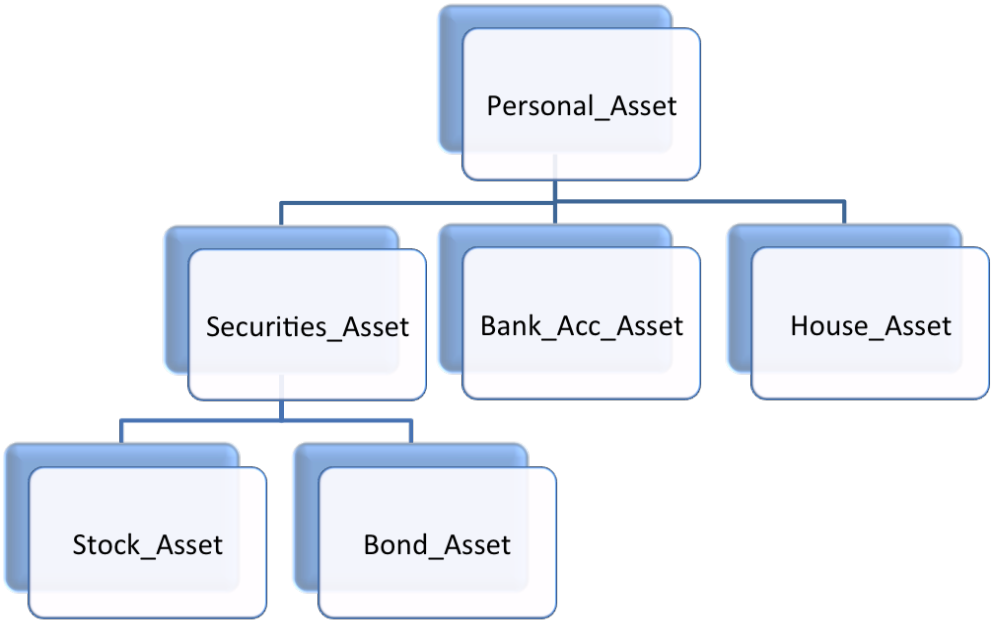
- Do not rely on the virtual function mechanism during the execution of a constructor.
- This is not a bug, but necessary — how can the derived object provide services if it has not been constructed yet?
- Similarly, if a virtual function is called inside the base class destructor, it represents base class' virtual function: when a derived class is being deleted, the derived-specific portion has already been deleted before the base class destructor is called!

Part VII

As Simple as ABC: Abstract Base Class

ABC Example: Assets

- Let's design a system for maintaining our assets: stocks, bank accounts, real estate, cars, yachts, etc.
- Each asset has a net worth (monetary value). We would like to be able to make listings and compute the total net worth.
- There are different kinds of assets, and they are all derived from **Personal_Asset**.



ABC Example: Personal_Asset + Bank_Acc_Asset Classes

```
class Personal_Asset    /* File: personal-asset.h */
{
  public:
    Personal_Asset(const string& date) : purchase_date(date) { }
    void set_purchase_date(const string& d);
    virtual double compute_net_worth() const; // Current net worth
    virtual bool is_insurable() const;        // Can this asset be insured?

  private:
    string purchase_date;
};
```

```
class Bank_Acc_Asset : public Personal_Asset /* File: bank-acc-asset.h */
{
  public:
    Bank_Acc_Asset(const string& d, double m, double r = 0.0)
      : Personal_Asset(d), balance(m), interest_rate(r) { }
    virtual double compute_net_worth() const override { return balance; }

  private:
    double balance;
    double interest_rate;
};
```

ABC Example: compute-assets.cpp

- There can be other classes of assets such as **Car_Asset**, **Securities_Asset**, **House_Asset**, etc.
- One may compute the total asset value for an array of different kinds of assets as follows:

```
/* File: compute-assets.cpp */
double compute_total_worth(const Personal_Asset* asset[], int num_assets)
{
    double total_worth = 0.0;

    for (int i = 0; i < num_assets; i++)
        total_worth += assets[i]->compute_net_worth();

    return total_worth;
}
```

ABC Example: Personal_Asset Class Implementation

- Now we have to implement the member functions of the base class **Personal_Asset**.
- How to implement **Personal_Asset::compute_net_worth( )**?
- It depends completely on the actual type of asset. There is no "standard way" of doing it!

```
/* File: personal-asset.cpp */
Personal_Asset::Personal_Asset(const string& date)
     : purchase_date(date) { }

void Personal_Asset::set_purchase_date(const string& date)
     { purchase_date = date; }

double Personal_Asset::compute_net_worth() const
     { return /* What? */ }
```

ABC Example: How to Implement compute_net_worth( ) ?

- The truth is: It makes no sense to have objects of type **Personal_Asset**.
- Such an object has only a purchase date, but otherwise no meaning. It is not a bank account, not a car, not a house — it is too general to be used.
- We cannot implement the **compute_net_worth( )** method in the base class **Personal_Asset** as the information needed to implement it is missing.
- However, we don't want to remove the method because that would make a polymorphic function like **compute_total_worth( )** impossible.

Solution: Abstract Base Class (ABC)

The solution is to make **Personal_Asset** an abstract base class (ABC), and **compute_net_worth( )** now becomes a pure virtual function.

```
class Personal_Asset    /* File: personal-asset-abc.h */
{
  public:
    Personal_Asset(const string& date) : purchase_date(date) { }
    void set_purchase_date(const string& d);
    virtual bool is_insurable() const; // Can this asset be insured?

    // A pure virtual function to compute the current net worth
    virtual double compute_net_worth() const = 0;

  private:
    string purchase_date;
};
```

Abstract Base Class (ABC)

```
Personal_Asset p_asset("1997/07/01");        // Error
Bank_Acc_Asset b_asset("2000/01/01", 100.0); // Ok
```

- An ABC has two properties:
  1. No objects of ABC can be created.
  2. Its derived classes must implement the pure virtual functions, otherwise they will also be ABC's.
- If a derived class, e.g., **Securities_Asset**, does not implement the pure virtual functions, then
  - the derived class is also an ABC, and
  - there cannot be objects of that type,
  - but it can be used as a base class itself, for instance for **Stocks_Asset**, **Bonds_Asset**, etc.

Interface Re-use

ABC as an Interface

An abstract base class provides a uniform interface to deal with a number of different derived classes.

- A base class contains what is common about several classes.
- If the only thing that is common is the interface, then the base class is a "pure interface," called ABC in C++.
- We discussed before that code re-use is an advantage of inheritance.
- For ABC's, we do not re-use code, but create an interface that can be re-used by its derived classes.
- Interfaces are the soul of object-oriented programming. They are the most effective way of separating the use and implementation of objects.
- The user (of **compute_total_worth( )**) only knows about the abstract interface, objects from different derived classes of the ABC may implement the interface in different ways.

Final Remarks on ABC

- A pure virtual function is inherited as a pure virtual function by a derived class unless it implements the function.
- An abstract base class cannot be used
  - as an argument type that is passed by value
  - as a function return type that is returned by value
  - as the type of an explicit conversion
- However, pointers and references to an ABC can be declared.
- Calling a pure virtual function from the constructor of an ABC is undefined — don't do that.

ABC Example: Do and Don't

```
#include <string>       /* File: can-and-cant.cpp */
using namespace std;

#include "personal-asset-abc.h"
#include "bank-acc-asset.h"

Personal_Asset x("20010/01/01");      // Error: can't create ABC object
Personal_Asset f1(int x) { /* .. */ } // Error: can't return ABC object
int f2(Personal_Asset x) { /* .. */ } // Error: can't CBV with ABC object

Bank_Acc_Asset b("01/01/2000", 0.0);  // OK!
Personal_Asset* p_asset_ptr = &b;     // OK!
Personal_Asset& p_asset_ref = b;      // OK!

Personal_Asset* f3(const Personal_Asset& x) { /* incomplete */ } // OK!
```

Part VIII

The C++11 Keyword final: No More Offspring

A final Class

```
1  #include <iostream>     /* File: final-class-error.cpp */
2  using namespace std;
3
4  class A {};
5  class B: public A {};
6  class C final: public B {};
7  class D: public B {};
8  class E: public C {};
9
10 int main()
11 {
12     A a; B b; C c; D d; E e;
13     return 0;
14 }
```

```
final-class-error.cpp:8:7: error: cannot derive from 'final' base 'C'
in derived type 'E'
    class E: public C {};
          ^
```

No sub-classes can be derived from a final class.

Example: No PG_Student if Student Class is final

```
1   #include <iostream>     /* File: pg-final-error.cpp */
2   using namespace std;
3
4   class UPerson { /* incomplete */ };
5   class Student final : public UPerson { /* incomplete */ };
6   class PG_Student : public Student { /* incomplete */ };
7
8   int main()
9   {
10      UPerson abby("Abby", CBME);
11      Student bob("Bob", CIVL, 3.0);
12      PG_Student matt("Matt", CSE, 3.8);
13  }
```

```
pg-final-class-error.cpp:6:7: error: cannot derive from 'final' base
'Student' in derived type 'PG_Student'
    class PG_Student : public Student { /* incomplete */ };
          ^~~~~~~~~~
```

Example: No PG_Student::print if Student::print is final

```
1  #include <iostream>     /* File: final-vfcn-error.cpp */
2  using namespace std;
3
4  class UPerson {
5    public: /* Other data and functions */
6      virtual void print() const { /* incomplete */ }
7  };
8
9  class Student : public UPerson {
10   public: /* Other data and functions */
11     virtual void print() const override final { /* incomplete */ }
12 };
13
14 class PG_Student : public Student {
15   public: /* Other data and functions */
16     virtual void print() const override { /* incomplete */ }
17 };
18
19 int main() { PG_Student jane("Jane", CSE, 4.0); jane.print(); }
```

Example: No PG_Student::print if Student::print is final ..

```
final-vfcn-error.cpp:16:18: error: virtual function
'virtual void PG_Student::print() const'
     virtual void print() const override { /* incomplete */ }
                  ^~~~~
final-vfcn-error.cpp:11:18: error: overriding final function
'virtual void Student::print() const'
     virtual void print() const override final { /* incomplete */ }
                  ^~~~~
```

Can't override a final virtual function.

Part IX

Further Reading: Public / Protected / Private Inheritance

"You are such a drama queen! Heaven knows where you get that from!"

Different Types of Inheritance

- So far, we have been dealing with only public inheritance.

```
class Student: public UPerson { ... }
```

- There are two other kinds of inheritance: protected and private inheritance.
- They control how the inherited members of Student are accessed by Student's derived classes (*not* by the Student class itself) or global functions.

UPerson Class Again

```
1  #ifndef UPERSON_H        /* File: uperson.h */
2  #define UPERSON_H
3
4  enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
5
6  class UPerson
7  {
8    private:
9      string name;
10     Department dept;
11
12   protected:
13     void set_name(string n) { name = n; }
14     void set_department(Department d) { dept = d; }
15
16   public:
17     UPerson(string n, Department d) : name(n), dept(d) { }
18     string get_name() const { return name; }
19     Department get_department() const { return dept; }
20 };
21 #endif
```

Student Class Again

```
1  #ifndef STUDENT_H        /* File: student.h */
2  #define STUDENT_H
3
4  #include "uperson.h"
5  class Course { /* incomplete */ };
6
7  class Student : ??? UPerson // ??? = public/protected/private
8  {
9    private:
10     float GPA;
11     Course* enrolled;
12     int num_courses;
13
14   public:
15     Student(string n, Department d, float x) :
16         UPerson(n, d), GPA(x), enrolled(nullptr), num_courses(0) { }
17     float get_GPA() const { return GPA; }
18     bool enroll_course(const string& c) { /* incomplete */ };
19     bool drop_course(const Course& c) { /* incomplete */ };
20 };
21 #endif
```

Example: Public Inheritance

```
class Student: public UPerson { ... }
```

public	protected	private
get_name( )	set_name( )	name
get_department( )	set_department( )	dept
get_GPA( )		GPA
enroll_course( )		enrolled
drop_course( )		num_courses

Example: Protected Inheritance

```
class Student: protected UPerson { ... }
```

public	protected	private
	set_name( )	name
	set_department( )	dept
get_GPA( )	get_name( )	GPA
enroll_course( )	get_department( )	enrolled
drop_course( )		num_courses

Example: Private Inheritance

```
class Student: private UPerson { ... }
```

public	protected	private
		name
		dept
get_GPA( )		GPA
enroll_course( )		enrolled
drop_course( )		num_courses
		set_name( )
		set_department( )
		get_name( )
		get_department( )

Slicing with Public Inheritance Again

Given the following definitions and public inheritance is used:

```
class Derived : public Base { ... }
Base base;
Derived derived;
```

- The following assignments are fine:

```
base = derived;      // Slicing
Base* b = &derived; // Can't use derived-class specific members
Base& b = derived;  // Can't use derived-class specific members
```

- The following assignments give compilation errors:

```
derived = base;      // Unless you define such conversion
Derived* d = &base; // No such conversion
Derived& d = base;  // No such conversion
```

No Slicing for Protected and Private Inheritance

If you use protected/private inheritance, slicing won't work either. That is, none of the assignments in the previous page work.

```
1  #include <string>        /* File: no-slicing.cpp */
2  using namespace std;
3
4  // class Student: protected UPerson { ... }
5  #include "protected-student.h"
6
7  int main()
8  {
9      Student ug("UG", ECE, 3.0);
10     UPerson p = ug;      // Allowed or not?
11     UPerson* q = &ug;    // Allowed or not?
12     UPerson& r = ug;     // Allowed or not?
13     return 0;
14 }
```

No Slicing for Protected and Private Inheritance ..

```
no-slicing.cpp:10:17: error: cannot cast 'const Student' to its
protected base class 'const UPerson'
    UPerson p = ug;     // Allowed or not?
                ^
./protected-student.h:7:17: note: declared protected here
class Student : protected UPerson
                ~~~~~~~~~~~~~~~~~
no-slicing.cpp:11:18: error: cannot cast 'Student' to its
protected base class 'UPerson'
    UPerson* q = &ug;   // Allowed or not?
                 ^
no-slicing.cpp:12:18: error: cannot cast 'Student' to its
protected base class 'UPerson'
    UPerson& r = ug;    // Allowed or not?
                 ^
```

Quiz: Why the first error mentions a 'const Student' instead of a 'Student'?

Inheritance: Summary

1. Public inheritance preserves the original accessibility of inherited members:

$$\begin{aligned} \text{public} &\Rightarrow \text{public} \\ \text{protected} &\Rightarrow \text{protected} \\ \text{private} &\Rightarrow \text{private} \end{aligned}$$

2. Protected inheritance affects only public members and renders them protected.

$$\begin{aligned} \text{public} &\Rightarrow \text{protected} \\ \text{protected} &\Rightarrow \text{protected} \\ \text{private} &\Rightarrow \text{private} \end{aligned}$$

3. Private inheritance renders all inherited members private.

$$\begin{aligned} \text{public} &\Rightarrow \text{private} \\ \text{protected} &\Rightarrow \text{private} \\ \text{private} &\Rightarrow \text{private} \end{aligned}$$

Inheritance: Summary ..

- The various types of inheritance control the highest accessibility of the inherited member data and functions.
- Public inheritance implements the "is-a" relationship.
- Private inheritance is similar to "has-a" relationship.
- Public inheritance is the most common form of inheritance.
- Private and protected inheritance do not allow casting of objects of derived classes back to the base class.

Question: Does polymorphism (by overriding) work with protected/private inheritance?

E.g., for the **UPerson** example, try to derive **Student** or **Teacher** by protected or private inheritance.